



Red Hat OpenShift AI Self-Managed 3.5

Customize Models for Gen AI and Agentic AI
Applications

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

Learn how to customize a model, from setting up your development environment to building and deploying a model specific to your domain-specific use case.

Table of Contents

PREFACE	4
CHAPTER 1. MODEL CUSTOMIZATION WORKFLOW	5
CHAPTER 2. SET UP YOUR WORK ENVIRONMENT	6
2.1. ABOUT THE RED HAT PYTHON INDEX	6
2.2. MIRROR THE PYTHON INDEX FOR YOUR DISCONNECTED ENVIRONMENT	6
2.3. INSTALL PACKAGES AND JUPYTERLAB	6
2.4. IMPORT EXAMPLE NOTEBOOKS	7
2.4.1. Clone an example Git repository	8
CHAPTER 3. PREPARE YOUR DATA FOR AI CONSUMPTION	10
3.1. PROCESS DATA BY USING DOCLING	10
3.2. EXPLORE THE DATA PROCESSING EXAMPLES	10
3.3. AUTOMATE DATA PROCESSING STEPS BY BUILDING AI PIPELINES	11
3.4. EXPLORE THE KUBEFLOW PIPELINE EXAMPLES	11
CHAPTER 4. GENERATE SYNTHETIC DATA	12
4.1. EXPLORE THE SDG HUB EXAMPLES	12
4.1.1. SDG Hub Knowledge Tuning example	13
4.1.2. SDG Hub Text analysis example for structured insights	14
4.1.3. SDG Hub RAG Evaluation example	14
4.1.4. SDG Hub Red Teaming & AI Safety example	14
4.1.5. SDG Hub Agentic examples	14
4.2. PERFORMANCE BENCHMARKS FOR KNOWLEDGE TUNING	15
4.3. GUIDED EXAMPLES - BUILD A KFP PIPELINE FOR SDG	15
4.3.1. Domain Customization Data Generation using Kubeflow Pipelines	15
4.3.2. Basic Kubeflow pipeline example	16
4.3.2.1. Customize the basic Kubeflow pipeline example	18
CHAPTER 5. GENERATE EVALUATION DATA FOR TOOL CALLING	20
5.1. MCP EVALUATION BENCHMARK PIPELINE	20
5.1.1. Pipeline architecture	20
5.1.2. Agent architecture and model swapping	20
5.1.3. Relationship to MCP server Distillation	21
5.1.4. SDG Hub components	21
5.1.5. Data sovereignty	21
5.2. SAFETY CONSIDERATIONS FOR MCP TOOL EXPLORATION	22
5.2.1. Active tool invocation risks	22
5.2.2. MCP tool annotations	22
5.2.3. Risk reduction practices for automated tool exploration	22
5.2.4. Credential security during tool exploration	23
5.3. MCP SERVER CREDENTIAL HANDLING	23
5.4. MCP EVALUATION PIPELINE CONFIGURATION OPTIONS	25
5.4.1. Startup script options	27
5.5. GENERATE EVALUATION DATA FROM CUSTOM MCP SERVERS	28
5.6. CONFIGURE QUALITY VALIDATION FOR EVALUATION DATA	31
5.7. MCP EVALUATION OUTPUT FORMAT AND SCORING DIMENSIONS	32
5.7.1. Benchmark tasks output schema	32
5.7.2. Evaluation results output schema	33
5.7.3. Programmatic trace metrics	33
5.7.4. LLM-as-judge scoring dimensions	34

- 5.7.5. Defect-rate calibration rubric 34
- 5.7.6. Failure case constants 35
- 5.8. MCP EVALUATION FRAMEWORK COMPATIBILITY 35
- CHAPTER 6. TRAIN THE MODEL BY USING YOUR PREPARED DATA 37
 - 6.1. EXPLORE TRAINING HUB EXAMPLES 37
 - 6.2. TRAINING HUB ALGORITHM AND MODEL SUPPORT MATRIX 37
 - 6.3. ESTIMATE MEMORY USAGE 38
 - 6.4. COMPARE THE PERFORMANCE OF OSFT, SFT, AND LORA TRAINING ALGORITHMS 39
 - 6.5. TRAINING HUB IN OPENSIFT AI 40
 - 6.6. TRACK EXPERIMENTS WITH MLFLOW AND TRAINING HUB 40
 - 6.6.1. Benefits of experiment tracking 41
 - 6.6.2. Enable MLflow tracking 41
 - 6.7. DISTRIBUTE TRAINING JOBS BY USING THE KUBEFLOW TRAINER 42
 - 6.7.1. Distributed fine-tuning with Training Hub and Kubeflow Trainer 42
- CHAPTER 7. IMPROVE MODEL ACCURACY WITH INFERENCE-TIME SCALING 43
 - 7.1. GENERATE COMPLETE RESPONSES 43
 - 7.2. STEP-BY-STEP REASONING 45
- CHAPTER 8. END-TO-END MODEL CUSTOMIZATION WORKFLOW 47
- CHAPTER 9. SUPPORT PHILOSOPHY: A SECURE PLATFORM 48

PREFACE

Learn how to customize a model, from setting up your development environment to building and deploying a model specific to your domain-specific use case.

CHAPTER 1. MODEL CUSTOMIZATION WORKFLOW

Red Hat AI model customization empowers you to tailor artificial intelligence models to your unique data and operational requirements. The model customization process involves the training or fine-tuning of pre-existing models with proprietary datasets, followed by their deployment with specific configurations on the Red Hat OpenShift AI platform. This comprehensive approach is facilitated by a powerful suite of integrated toolkits that streamline and accelerate the development of generative AI applications.

The workflow for customizing models includes the following tasks:

Set up your working environment

Ensure reliable and secure access to supported libraries with the Red Hat Hosted Python index. For details, see [Set up your working environment](#).

Prepare your data for AI consumption

To prepare your data, use Docling, a powerful Python library to transform unstructured data (such as text documents, images, and audio files) into structured formats that models can consume. For details, see [Prepare your data for AI consumption](#).

To automate data processing tasks, you can build Kubeflow Pipelines (KFP), see [Automate data processing steps by building AI pipelines](#).

Generate synthetic data

Use the Red Hat AI Synthetic Data Generation (SDG) Hub framework to build, compose, and scale synthetic data pipelines with modular blocks. With the SDG Hub, you can extend your synthetic data pipelines with custom blocks to fit your domain, replace ad hoc scripts with the SDG Hub repeatable framework, and scale data generation with asynchronous execution and monitoring. For details, see [Generate synthetic data](#).

Train a model by using your prepared data

After you prepare your data, use the Red Hat AI Training Hub to simplify and accelerate the process of fine-tuning and customizing a foundation model by using your own data.

You can extend a base notebook to use distributed training across many nodes by using the Kubeflow Trainer. The Kubeflow Trainer abstracts the underlying infrastructure complexity of distributed training and fine-tuning of models. The iterative process of fine-tuning significantly reduces the time and resources required compared to training models from scratch.

For details, see [Train a model by using your prepared data](#).

Improve model accuracy with inference-time scaling

You can use Inference-time scaling (ITS) to improve the quality of model responses by allocating additional compute at inference time. Instead of generating a single response to a prompt, the model generates multiple candidate outputs and uses a selection strategy to return the best one. This approach produces better results without modifying model weights, at the cost of increased compute per query. For details about the scaling algorithms that the ITS Hub library provides, see [Improve model accuracy with inference-time scaling](#).

Serve and consume a customized model

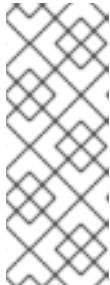
After you customize a model, you can serve your customized models as APIs (Application Programming Interfaces). Serving a model as an API enables seamless integration into existing or newly developed applications.

Learn more about serving and consuming a customized model in [Deploying models on the model serving platform](#).

CHAPTER 2. SET UP YOUR WORK ENVIRONMENT

To set up your working environment for customizing models, complete these tasks:

1. For disconnected environments, mirror the Python index.
2. Start a JupyterLab workbench that is pre-configured to use the Red Hat Python index to run example notebooks.



NOTE

The default workbenches provided with the Red Hat OpenShift AI installation are pre-configured to use the Red Hat Python index, except for the **Jupyter | TensorFlow | ROCm | Python 3.12** image. Instead, you must create a custom workbench image based on the following base image:

[Red Hat Ecosystem catalog base image for rhai/base-image-rocm-6.4-rhel9](#)

3. From your running workbench, import example notebooks.

2.1. ABOUT THE RED HAT PYTHON INDEX

Red Hat AI includes a maintained Python package index that provides secure and reliable access to supported libraries, with full support for disconnected environments.

For a list of packages in an index for a specific OpenShift AI release and runtime, go to the [Red Hat AI Components Indexes](#) page.

For details about Red Hat support for the Python package index, see [Support philosophy: A secure platform](#).

2.2. MIRROR THE PYTHON INDEX FOR YOUR DISCONNECTED ENVIRONMENT

If you are using a disconnected environment, use the following code example to access the Red Hat Python index content and copy it locally. You can then upload the packages into your own internal hosting service:

```
#!/bin/bash -x

URL=https://packages.redhat.com/api/pypi/public-rhai/rhoai/3.4/cuda12.9-ubi9/simple/
wget \
  --verbose \           # Show detailed progress
  --mirror \           # Mirror the directory structure
  --continue \         # Resume partial downloads
  --no-host-directories \ # Don't create host-named directories
  --cut-dirs=4 \       # Remove first 4 path segments
  $URL
```

2.3. INSTALL PACKAGES AND JUPYTERLAB

To ensure reliable and secure access to supported libraries, start your model customization workflow with a JupyterLab workbench that is pre-configured with the Red Hat Python index.

When you use one of the images, both **pip** and **uv** commands are pre-configured to use the Red Hat Python index and system trust store for HTTPS.

When you run a **pip install** command, it installs the package version referenced in the Red Hat Python index, ensuring that you are installing a version of the library that is secure and reliably accessible.

For example, use the following commands to install the model customization libraries:

- Install the data processing library:

```
pip install docling
```

- Install the synthetic data generation library:

```
pip install sdg-hub
```

- Install the model training library:

```
pip install training-hub
```

Install the model training library with CUDA support:

```
pip install training-hub[cuda]
```

For additional options and details for installing the model training library, see [Training Hub installation guidelines](#).

- Install the inference time scaling library:

```
pip install its-hub
```

2.4. IMPORT EXAMPLE NOTEBOOKS

To get started with customizing your models, you can run provided example notebooks and scripts. The following table lists the Git repositories that provide example notebooks for each model customization component.

For a comprehensive tutorial that demonstrates an AI/ML workflow, see the [Knowledge Tuning example](#) on the Red Hat AI examples site.

The Knowledge Tuning tutorial is a curated collection of Jupyter notebooks that includes examples of using Docling to process data, Training Hub to fine-tune a model on that data, and KServe to deploy the final model for a Question and Answer application.

Table 2.1. Model customization example notebooks

Model customization component	Git clone example repository	Branch	Directory
Data processing using Docling	https://github.com/opendatahub-io/data-processing.git	stable	notebooks/

Model customization component	Git clone example repository	Branch	Directory
Synthetic data generation	https://github.com/Red-Hat-AI-Innovation-Team/sdg_hub.git	main	examples
Training	https://github.com/Red-Hat-AI-Innovation-Team/training_hub.git	main	examples
End-to-end example for model customization with these components	https://github.com/red-hat-data-services/red-hat-ai-examples.git	main	knowledge-tuning

2.4.1. Clone an example Git repository

Follow these steps to clone a Git repository from the JupyterLab environment provided with your OpenShift AI workbench.

Prerequisites

- You have the https URL and branch for one of the example Git repositories listed in Table 2.2.

Procedure

- From the OpenShift AI dashboard, go to the project where you created a workbench.
- Click the link for your workbench. If prompted, log in and allow JupyterLab to authorize your user.
Your JupyterLab environment window opens.

The file-browser window shows the files and directories that are saved inside your own personal space in OpenShift AI .

- Bring the content of an example Git repo inside your JupyterLab environment:
 - On the toolbar, click the **Git Clone** icon.
 - Enter a Git https URL.
 - Select the **Include submodules** option, and then click **Clone**.
- If you want to use a branch other than **main** (for example, the data processing example repo uses the **stable-3.0** branch), change the branch:
 - In the left navigation bar, click the **Git** icon, and then click **Current Branch** to expand the branches and tags selector panel.
 - On the **Branches** tab, in the **Filter** field, enter the branch name.
 - Select the branch.

The current branch changes to the branch that you selected.

Verification

- In the file browser, double-click the newly-created directory to see the example files.

CHAPTER 3. PREPARE YOUR DATA FOR AI CONSUMPTION

To prepare your data, use Docling to transform unstructured data (such as text documents, images, and audio files) into structured formats that models can consume.

To automate data processing tasks, you can build KubeFlow Pipelines (KFP). For examples of pre-built pipelines for unstructured data processing with Docling, see <https://github.com/opendatahub-io/data-processing>.

3.1. PROCESS DATA BY USING DOCLING

Docling is the Python library that you use to prepare unstructured data (like PDFs and images) for consumption by large language models.

3.2. EXPLORE THE DATA PROCESSING EXAMPLES

To get started with data processing with Docling, explore the provided examples.

Prerequisites

- Install the data processing library as described in [Set up your working environment](#).

Procedure

1. To access the data processing examples, use one of the following methods to clone the [data processing Git repository](#):

- To clone the <https://github.com/opendatahub-io/data-processing.git> repository from JupyterLab, follow the steps in [Clone an example Git repository](#) and specify the **stable** branch.
- To create a local clone of the repository, run the following command:

```
git clone https://github.com/opendatahub-io/data-processing -b stable
```

2. Go to the [notebooks](#) directory to learn how to use Docling for the following tasks:
[Use cases](#)

- Convert - Change unstructured documents (PDF files) to structured format (Markdown), with and without vision-language model (VLM)
- Chunk - Split documents into smaller, semantically meaningful pieces
- Extract information - Use template formats to extract specific data fields from documents like invoices.
- Select subsets - Reduce the size of your dataset. The algorithm analyzes an input dataset and reduces it in size, while ensuring data diversity and coverage.

Tutorials - An example notebook that provides a complete, end-to-end workflow for preparing a dataset of documents for a RAG (Retrieval-Augmented Generation) system.

Additional resources

- Docling community project: <https://docling-project.github.io/docling/>
- GitHub Repository for the Docling project source code: <https://github.com/docling-project/docling>

3.3. AUTOMATE DATA PROCESSING STEPS BY BUILDING AI PIPELINES

With Kubeflow Pipelines (KFP), you can automate complex, multi-step Docling data processing tasks into scalable workflows.

With the KFP Software Development Kit (SDK), you can define custom components and stitch them together into a complete pipeline. The SDK allows you to fully control and automate Docling conversion tasks with specific parameters.

Note: You can build a custom runtime image to ensure that all required Docling dependencies are present for pipeline execution. For information on how to run a Docling pipeline with a custom image see the [Docling Pipeline documentation](#).

3.4. EXPLORE THE KUBEFLOW PIPELINE EXAMPLES

To get started with Kubeflow Pipelines, explore the provided examples. You can download and modify the example code to quickly create a Docling data processing or model training pipeline.

Prerequisites

- Install the data processing library as described in [Set up your working environment](#).

Procedure

1. To access the [Kubeflow Pipeline examples](#), run the following command to clone the [data processing Git repository](#):

```
git clone https://github.com/opendatahub-io/data-processing -b stable
```

2. Go to the **kubeflow-pipelines** directory, which contains the following tested examples for running Docling as a scalable pipeline. For instructions on how to import, configure, and run the examples, see the [README file](#) and the Red Hat AI [Working with AI pipelines](#) guide.
 - Standard Pipeline: For converting standard documents that contain text and structured elements. For more information, see the [Standard Conversion Pipelines documentation](#).
 - VLM (Vision Language Model): For converting highly complex or difficult-to-parse documents, such as those with custom instructions or complex layouts, or to add image descriptors. For more information, see the [VLM Pipelines documentation](#).

NOTE: If you want to use a Red Hat container image in one of the above pipelines, replace the image on [this line](#) with [the URL for the Red Hat Docling container image](#) and recompile the pipeline that you want to use.

CHAPTER 4. GENERATE SYNTHETIC DATA

When you customize a model for your enterprise, you must generate high-quality synthetic data to augment your data set, improve model robustness, and cover edge cases.

Red Hat provides the Synthetic Data Generation (SDG) Hub, a modular Python framework for building synthetic data generation pipelines by using composable blocks and flows. Each block performs a specific task, such as LLM chat, parse text, evaluate, or transform data. Flows chain blocks together to create complex data generation pipelines that include validation and parameter management. A flow (data generation pipeline) is a YAML specification that defines an instance of a data generation algorithm.

Additional resources

- [SDG community documentation/website](#)
- [SDG GitHub repository](#)

4.1. EXPLORE THE SDG HUB EXAMPLES

To get started with SDG Hub, explore the provided examples.

Prerequisites

- Install the Synthetic Data Generation (SDG) Hub library as described in [Set up your working environment](#).

Procedure

1. To access the SDG Hub examples, clone the [SDG Hub Git repository](#):
 - To clone the https://github.com/Red-Hat-AI-Innovation-Team/sdg_hub.git repository from JupyterLab, follow the steps in [Clone an example Git repository](#).
 - To create a local clone of the repository, run the following command:

```
git clone https://github.com/Red-Hat-AI-Innovation-Team/sdg_hub
```

2. Go to the [examples](#) directory to view the notebooks and YAML files for the use cases described in the following sections.

Each use case directory includes a **README.md** file that provides details for each use case, such as instructions, performance notes, and configuration tips.

When you run the example notebooks, consider the following information:

- **Data generation time and statistics:** The total time to generate data depends on both the maximum concurrency supported by your endpoint and the complexity of the running flow. Longer flows, such as the flows in the Knowledge Generation notebooks, take more time to complete because they produce a large number of summaries and Q&A pairs, each of which undergoes verification within the pipeline.
- **LLM endpoint requirements:** For running flows in the Knowledge Generation notebooks, Red Hat recommends that you set the following values:

• **NUMBER OF SUMMARIES:** a minimum of 10

- Set **NUMBER_OF_SUMMARIES** to a minimum of 10.
- To achieve reasonable data generation times and avoid timeouts, use an endpoint that supports a maximum concurrency of at least 50.
- Extend LiteLLM's request timeout by setting the environment variable **LITELLM_REQUEST_TIMEOUT**.

4.1.1. SDG Hub Knowledge Tuning example

The [knowledge tuning pipeline example](#) shows how to generate synthetic training data from enterprise documents, so that a fine-tuned model can accurately recall domain-specific content and facts in response to user queries.

Pre-trained language models typically encounter most facts only once or twice during training, leaving proprietary or domain-specific knowledge incomplete or absent. The knowledge tuning pipeline example addresses this problem by generating **n** augmentations per document, where **n** is a configurable parameter (**NUMBER_OF_SUMMARIES**), and converting each augmentation into synthetic question and answer (Q&A) pairs. Increasing summary token count leads to superior memorization.

This pipeline example implements the following augmentation instances:

- Thematic Summaries for capturing high-level ideas.
- Knowledge Relationships for identifying connections between segments.
- Atomic Facts for isolating granular details.

For each summary, the system generates three Q&A pairs; it discards excess pairs during post-processing. This architecture is modular. You can integrate additional augmentation types by editing existing flows or by adding new ones.

The output is a high-quality JSONL data set that is ready for consumption through training. This example provides a complete walkthrough of data generation and preparation for training, from document parsing through data set combination.

For benchmark results, token scaling statistics, and custom domain evaluations, see the [SDG Hub Knowledge Tuning example repo](#).

Multilingual knowledge tuning example

The knowledge generation pipeline also includes a [multilingual knowledge tuning example](#) that supports creating training data in any language. The example includes pre-built flows for Spanish that are auto-discovered with zero configuration. For other languages, the library automatically translates flows on demand by using an LLM, with a verification pass to ensure accuracy.

To get started, set **SDG_LANG** (for example, **Spanish**) and **SDG_LANG_CODE** (for example, **es**) in your environment, then run the knowledge generation notebook as normal. The notebook checks for a pre-translated flow variant and uses it if available. If no pre-translated flow exists, the notebook translates all prompt templates, verifies them with a second LLM pass, and registers the new flow for immediate use with no extra code.

Performance benchmarks on the QuALITY benchmark (translated to Spanish) show that multilingual fine-tuning improves domain-specific accuracy while preserving general model capabilities. SFT on Spanish synthetic data improved benchmark accuracy from a 44.47% baseline to 68.88% in RAG

settings, while retaining 84–90% of the baseline model’s behavior. For configuration details, benchmark tables, and coverage analysis, see the [multilingual documentation](#).

4.1.2. SDG Hub Text analysis example for structured insights

The [text analysis example](#) shows how to generate data for teaching models, to extract meaningful insights from text in structured format. Create custom blocks and extend existing flows for new applications.

4.1.3. SDG Hub RAG Evaluation example

Reliable, repeatable evaluation is essential before you ship RAG or agentic updates to production. The [RAG Evaluation example](#) provides a flow for generating data for evaluating RAG systems at scale. The input of the flow is user documents. The output data set is post-processed to work with the RAGAS framework. The flow creates Q&A pairs with ground truth context for evaluating RAG systems. This flow simplifies data generation for RAG workflows.

4.1.4. SDG Hub Red Teaming & AI Safety example

The [Teaming & AI Safety example](#) shows how to generate adversarial prompts for testing model safety, guardrails, and robustness against harmful content. It uses a challenger LLM as teacher. This example demonstrates how to create custom safety testing datasets based on your organization’s policies and concerns. The workflow generates harmful prompts with explanations. You can integrate it with security testing tools, such as Garak for automated safety validation on Red Hat OpenShift AI.

4.1.5. SDG Hub Agentic examples

The [agentic examples](#) demonstrate training data generation for custom MCP servers and agent tool-calling.

SDG Hub enables the creation of training data for AI agents and tool-calling applications through the following two complementary capabilities: generating tool-use training data from real tool interactions (MCP Server Distillation), and evaluating fine-tuned models through the same agent stack used for data generation (MCP Server Evaluation).

- [MCP Server Distillation](#) - SDG Hub can target any MCP server, and produce structured function-calling conversations ready for supervised fine-tuning. A frontier model actively explores the MCP server’s tools, calling every tool, discovering real data, mapping tool relationships and generating training examples grounded in what it found. The pipeline runs through 4 stages:
 - Expert exploration
 - Question synthesis
 - Quality filtering
 - Expert trajectory generation
- [MCP Server Evaluation](#) - After fine-tuning, to evaluate whether it worked, the evaluation pipeline uses a frontier model to generate gold-standard evaluation tasks at varying complexity levels on any custom MCP server that the input agent has access to. Use cases include training smaller models to use proprietary enterprise APIs and databases, reducing inference costs by

replacing expensive frontier models with fine-tuned alternatives, generating expert-quality tool-calling demonstrations at scale, and building domain-specific AI agents for custom tool ecosystems.

Agent Connector Framework

The Agent Connector Framework provides a standardized way to incorporate external tools, agents, APIs, and services into SDG pipelines. Instead of writing custom integration code for each tool, the connector framework offers a pluggable architecture that handles tool-specific communication while SDG Hub orchestrates execution at scale.

The Agent Connector Framework has the following capabilities:

- * Pluggable connector architecture for any external tool or service
- * Async orchestration with automated execution at scale
- * Production-ready error handling and retry logic
- * Extraction of results from tool interactions to generate rich, behavior-driven training data
- * Positioning of SDG Hub as a universal orchestration layer for multi-tool workflows, enabling AI engineers to build extensible data generation pipelines that incorporate proprietary tools and third-party services
- * Existing integrations, such as Langflow and Langgraph

Additional resources

- SDG community documentation: <https://github.com/instructlab/sdg/tree/main/docs>
- SDG GitHub repository: <https://github.com/instructlab/sdg>

4.2. PERFORMANCE BENCHMARKS FOR KNOWLEDGE TUNING

To get an estimate of the total time a flow will take, you can run the **dry_run** function and set **enable_time_estimation** to **true**.

For example, tests that use the gpt-oss-120b LLM on 4x H100 GPUs with the QuALITY dataset (266 articles) showed significant variance between flows.

- The estimated generation times for the full dataset were approximately 15.12 hours for Extractive Summary and 12.99 hours for Detailed Summary, both of which were evaluated with 50 completions per summary (N=50).
- In contrast, the Key Facts and Document Based flows, which generated only a single summary per document, completed in approximately 0.35 and 0.44 hours, respectively.
- Additionally, analysis of the Extractive Summary flow highlights that the steepest time reductions occurred between concurrency levels 10 and 30, with returns observed to diminish significantly beyond 50 in this specific configuration.

To view a graph that illustrates the accuracy on QuALITY Benchmark (4,609 Evaluation QA), go to: https://github.com/Red-Hat-AI-Innovation-Team/sdg_hub/blob/main/examples/knowledge_tuning/enhanced_summary_knowledge_tuning/imgs/qu

4.3. GUIDED EXAMPLES - BUILD A KFP PIPELINE FOR SDG

You can generate synthetic data for domain-specific model customization by using a Kubeflow Pipeline (KFP) on Red Hat OpenShift AI, as shown in the following examples.

4.3.1. Domain Customization Data Generation using Kubeflow Pipelines

The [Domain Customization Data Generation using Kubeflow Pipelines \(KFP\) example](#) demonstrates how to generate synthetic data for domain-specific model customization by using Kubeflow Pipelines.

Prerequisites

- Install the Synthetic Data Generation (SDG) Hub library as described in [Set up your working environment](#).

Procedure

1. Run the following command to clone the (org-name) AI examples repository that includes the KFP pipeline for knowledge tuning example.

```
git clone https://github.com/red-hat-data-services/red-hat-ai-examples
```

2. Navigate to the **examples/domain_customization_kfp_pipeline** directory.
3. Follow the instructions in the README file to run the example:
 - a. Configure an environment variable (**.env**) file, provide your model endpoint, and store the file as a Kubernetes secret. The KFP pipeline consumes the secret as environment variables.
 - b. Generate the KFP pipeline YAML file.
 - c. Upload the YAML file to OpenShift AI and deploy the pipeline.

Verification

The example pipeline generates three types of document augmentations and four types of QA on top of 3 augmentation and 1 original document. It stores the generated data in the Cloud Object Storage (COS) bucket that is linked through the pipeline server.

4.3.2. Basic Kubeflow pipeline example

The Kubeflow Pipelines guided example uses a minimal test pipeline to run a simple three-block flow that generates a question from each input document.

You can replace this flow with your own to perform more complex data generation tasks, such as multi-step augmentation, knowledge Q&A generation, or domain-specific transformations.

Prerequisites

- Access to an OpenShift AI cluster with Kubeflow Pipelines installed.
- A deployed LLM endpoint compatible with the LiteLLM format, for example: openai/gpt-4o-mini .
- The kfp Python SDK (version 2.15.2 or later) installed locally for pipeline compilation.

Procedure

1. Clone the Kubeflow Pipelines Components repository that includes the SDG component and pipeline:

```
$ git clone https://github.com/kubeflow/pipelines-components
```

2. Navigate to the **pipelines/data_processing/sdg** directory.
3. Prepare Kubernetes resources for the pipeline:
 - a. Create a ConfigMap containing your SDG flow YAML definition (the flow defines the sequence of data processing and LLM blocks to execute):

```
$ oc create configmap sdg-llm-test-flow --from-file=llm_test_flow.yaml
```

- b. Create a ConfigMap containing any prompt template files referenced by the flow:

```
$ oc create configmap sdg-llm-test-prompt --from-file=prompts/
```

- c. Create a Kubernetes Secret with your LLM API key:

```
$ oc create secret generic llm-credentials --from-literal=api_key=<your-api-key>
```

4. Compile the KFP pipeline YAML file:

```
$ python pipeline.py
```

The output of this command is the **sdg_llm_pipeline.yaml** file.

5. Upload the compiled **sdg_llm_pipeline.yaml** file to OpenShift AI and deploy the pipeline. When creating a pipeline run, configure the following parameters:

Parameter	Default	Description
model	openai/gpt-4o-mini	LiteLLM model identifier for your LLM endpoint.
max_concurrency	1	Maximum concurrent LLM requests.
temperature	0.7	LLM sampling temperature (0.0-2.0).
max_tokens	256	Maximum response tokens per LLM call.

The example pipeline uses **llm_test_flow.yaml**, a minimal three-block flow designed to demonstrate the SDG component end-to-end. It takes a JSONL data set with document and domain columns and generates a question for each document:

Block	Type	Purpose
build_question_prompt	PromptBuilderBlock	Reads the document column and builds a chat prompt using a template (prompts/generate_question.yaml) that instructs the LLM to generate a short question about the text.

Block	Type	Purpose
generate_question	LLMChatBlock	Sends the constructed prompt to the configured LLM and captures the raw response.
extract_question	LLMResponseExtractorBlock	Extracts the question text from the LLM response into a clean output column.

The prompt template uses a system message ("You are a helpful assistant that generates a short question about the given text") and a user message that inserts the **{{document}}** content.

Verification

The example pipeline executes two stages:

1. **Create sample data** - Generates a small JSONL data set with three rows containing document and domain columns.
2. **Run SDG flow** - Executes the three-block flow above against the sample data using the configured LLM endpoint.

When complete, the pipeline produces a KFP data set artifact containing the generated synthetic data in JSONL format (original columns plus the generated question columns), along with execution metrics (input rows, output rows, and execution time). If the pipeline server is linked to Cloud Object Storage, the artifacts are persisted there.

4.3.2.1. Customize the basic Kubeflow pipeline example

You can customize the basic Kubeflow pipeline example:

Use a different flow

Replace the flow YAML with your own. A flow is a YAML file that defines a sequence of blocks to execute on your data. You can chain together any combination of blocks supported by SDG Hub.

Use a built-in flow from the SDG Hub registry

Instead of mounting a custom flow YAML, you can reference a built-in flow by its registry ID. In the **pipeline.py** file, replace **flow_yaml_path** with **flow_id**:

```
sdg_task = sdg(
    input_artifact=data_task.outputs["output_data"],
    flow_id="your-builtin-flow-id",
    model=model,
    max_concurrency=max_concurrency,
    temperature=temperature,
    max_tokens=max_tokens,
)
```

When using a built-in flow, you do not need the flow ConfigMap.

Bring your own input data

The example pipeline generates sample data inline. To use your own data:

From a PVC: Mount a PersistentVolumeClaim and pass the path via the **input_pvc_path** parameter instead of **input_artifact**.

From an upstream component: Connect any KFP component that outputs a dsl.data set artifact in JSONL format to the input_artifact input of the SDG component.

Your input data must include the columns required by your chosen flow. These columns are defined in the flow's data set_requirements.required_columns.

Override parameters at runtime

Use runtime_params to override individual block parameters without modifying the flow YAML:

```
sdg_task = sdg(
    ...,
    runtime_params={"generate_question": {"temperature": 0.9, "max_tokens": 512}},
)
```

Enable checkpointing for large data sets

For long-running flows, enable checkpointing to allow resuming from the last saved state:

```
sdg_task = sdg(
    ...,
    checkpoint_pvc_path="/mnt/checkpoints/my-run",
    save_freq=50,
)
```

Export output to a PVC

In addition to the KFP artifact, you can export the generated data to a mounted PVC for direct access:

```
sdg_task = sdg(
    ...,
    export_to_pvc=True,
    export_path="/mnt/output",
)
```

The output is saved to **<export_path>/<flow_name>/<timestamp>/generated.jsonl**.

Additional resources

- [SDG Hub documentation](#) - Full library documentation including available flows and block types.
- [SDG component README](#) - Complete component parameter reference.
- [SDG pipeline README](#) - Pipeline metadata and dependencies.

CHAPTER 5. GENERATE EVALUATION DATA FOR TOOL CALLING

You can automatically generate tool-calling evaluation benchmark data from your custom Model Context Protocol (MCP) servers and evaluate models against those benchmarks. The evaluation pipeline produces verified question-answer-tool-call triplets with quality scoring. This allows you to rank models by their tool-calling accuracy on your tool ecosystem before deploying to production.

This capability extends the MCP Server Distillation workflow for training data generation. After you fine-tune a model on MCP distillation data, you use this evaluation pipeline to validate the model's performance with verified benchmarks.

5.1. MCP EVALUATION BENCHMARK PIPELINE

You can automatically generate tool-calling evaluation benchmark data from your custom Model Context Protocol (MCP) servers by using the Synthetic Data Generation (SDG) Hub evaluation pipeline. The pipeline produces verified question-answer-tool-call triplets that you can use to measure model accuracy and agent reliability.

5.1.1. Pipeline architecture

The evaluation data generation pipeline operates in four stages, building on the existing **AgentBlock** infrastructure that SDG Hub provides for connecting to already-deployed LangGraph agent endpoints that handle MCP server connectivity and tool invocation.

MCP exploration

The pipeline connects to your MCP servers, checks available tools using **tools/list**, discovers tool schemas with input and output types, and actively invokes tools by using a frontier LLM, a highly advanced and large-scale LLM, to discover real data and behavioral patterns. This produces evaluation data grounded in tool usage. The exploration stage captures tool dependencies and multi-step interaction patterns, for example, when tool A returns IDs that tool B requires.

Question generation

By using the exploration results, the pipeline generates evaluation questions at configurable complexity levels. The **num_samples** parameter directly controls how many tools each generated scenario must use: 2 for simple scenarios, 4 for moderate multi-tool sequences, and 8 for complex tool-chaining tasks. If a server has fewer tools than the **num_samples** value, the pipeline automatically skips that complexity level for that server.

Ground truth generation

For each generated question, the pipeline executes the expected tool-call sequence against the MCP server by using a frontier LLM to capture verified ground truth answers. This produces triplets that consist of a question, the expert tool trace with name, input, and output for each tool call, and the expert answer. The frontier LLM orchestrates the tool calls through a LangGraph agent, and the pipeline records both the raw tool responses and the synthesized answer.

Quality validation

The pipeline applies a two-tier quality validation approach that combines trace metrics for objective tool-call correctness with LLM-as-judge scoring. The judge evaluates each example across six dimensions: task fulfillment, grounding, tool appropriateness, parameter accuracy, dependency awareness, and parallelism and efficiency. Examples that fail quality thresholds are automatically filtered out.

5.1.2. Agent architecture and model swapping

A key architectural feature of the pipeline is that the same LangGraph agent is used for both data generation and evaluation. During data generation, the agent uses a frontier model as the teacher to produce high quality tool traces. During evaluation, the underlying LLM is swapped to a candidate model by using the LangGraph configurable parameter, while the full agent stack, including tools, guardrails, and orchestration, remains identical. This approach ensures that evaluation scores reflect the candidate model's tool-calling accuracy in isolation, without introducing variables from differences in agent configuration.

5.1.3. Relationship to MCP server Distillation

The evaluation pipeline extends the MCP Server Distillation capability that generates training data from custom MCP servers. Both capabilities share the same **AgentBlock** infrastructure and LangGraph connector, but serve different purposes. For more information about MCP Server Distillation, see [Generate synthetic data](#).

An example workflow is to generate training data with MCP Server Distillation, fine-tune a model on that data, and then generate evaluation benchmarks with this pipeline to validate the fine-tuned model's performance before production deployment.

5.1.4. SDG Hub components

The evaluation pipeline uses several SDG Hub components.

Flows

The MCP Server Distillation flow is a 23-block pipeline that handles MCP exploration, question generation, and ground truth generation. The Agent Tool-Use Evaluation flow is a 4-block pipeline that scores candidate models against the generated benchmarks.

Blocks

The pipeline uses **AgentBlock** to connect to already-deployed LangGraph agent endpoints that handle MCP server connectivity and tool invocation, and **AgentResponseExtractorBlock** to parse agent responses into structured evaluation data.

LangGraph connector

The Agent Connector Framework provides the standardized integration layer between SDG Hub and LangGraph agents, handling tool-specific communication and async orchestration.

5.1.5. Data sovereignty

The default pipeline configuration uses external OpenAI endpoints for the teacher and judge models. MCP server data and generated evaluation datasets remain within your infrastructure, but LLM inference requests are sent to the configured model endpoints.

You can achieve full data sovereignty by replacing the default external endpoints with on-platform options:

- The frontier LLM endpoint is configurable. You can use on-platform options such as KServe-served models or Models-as-a-Service instead of external OpenAI endpoints.
- MCP servers run locally within your infrastructure. Tool exploration data and generated evaluation datasets remain in your storage, whether local filesystem, S3, or a **PersistentVolumeClaim** (PVC).
- No external evaluation services are connected during pipeline execution. The only external traffic is to the configured LLM endpoints.

5.2. SAFETY CONSIDERATIONS FOR MCP TOOL EXPLORATION

The evaluation benchmark pipeline actively invokes tools on your MCP servers during exploration. You must understand the implications of automated tool invocation before running the pipeline against your MCP servers.

5.2.1. Active tool invocation risks

Unlike approaches that only read tool schemas, the evaluation pipeline actively calls tools on the target MCP servers to discover data and behavioral patterns. This active exploration is necessary to produce grounded evaluation data, but it introduces the risk that tools with side effects can modify data or system state.

Automated tool exploration can invoke the following types of operations, which pose risks in production environments:

- Database record creation, updates, or deletions
- Notification triggers, such as sending emails or creating tickets
- Infrastructure state modifications, such as scaling resources or changing configurations
- Financial transactions or state-changing API calls



IMPORTANT

Always run the evaluation pipeline against development or staging MCP servers before using it against production systems.

5.2.2. MCP tool annotations

The MCP protocol defines tool annotations that indicate tool behavior. You can use these annotations to identify tools that are safe for automated exploration.

readOnlyHint

Indicates that invoking the operation does not modify any state. Operations annotated with **readOnlyHint: true** are safe for automated exploration.

destructiveHint

Indicates that the operation can delete or permanently modify data. Operations annotated with **destructiveHint: true** require careful review before automated invocation.

idempotentHint

Indicates that invoking the operation multiple times with the same input produces the same result without additional side effects.

openWorldHint

Indicates that the operation interacts with external entities outside the server's control.

Not all MCP servers provide these annotations. When annotations are absent, treat all tools as potentially side-effecting until you verify their behavior.

5.2.3. Risk reduction practices for automated tool exploration

Follow these practices to reduce risk during automated MCP tool exploration:

- Always run the pipeline against development or staging MCP servers before using it against production systems.
- Review the list of discovered tools in the generation notebook output before proceeding to question generation.
- Exclude tools that create, update, or delete data unless your development environment is designed to handle automated modifications.
- Use dedicated test data in your MCP servers so that pipeline-generated tool calls do not affect real business data.
- Monitor MCP server logs during pipeline execution to detect unexpected state changes.



NOTE

The pipeline does not implement pre-execution tool safety classification filtering. You are responsible for reviewing and scoping tools before running the pipeline.

5.2.4. Credential security during tool exploration

The pipeline requires credentials to access both the frontier LLM endpoint and your MCP servers. Handle these credentials securely by using environment variables loaded from **.env** files rather than hardcoding values in notebooks. For production deployments, use Kubernetes Secrets or the Connection API for credential management.

5.3. MCP SERVER CREDENTIAL HANDLING

You must configure credentials for the frontier LLM endpoint and, optionally, for your MCP servers and LangGraph agents. Follow secure credential management practices to avoid exposing secrets in notebooks or pipeline output.

Required credentials

Table 5.1. Credentials used by the evaluation pipeline

Credential	Required	Description
OPENAI_API_KEY	Yes	API key for the frontier LLM endpoint used by both the teacher and judge models.
LANGGRAPH_API_KEY	No	API key for authenticating with LangGraph agent endpoints. Required only if your agents are configured with authentication.
Per-model API keys in MODEL_CONFIGS	No	API keys for candidate model endpoints during evaluation. Required when candidate models use authenticated endpoints.

Secure credential patterns

Store credentials by using one of the following methods, depending on your deployment environment.

Environment variables through .env files

For notebook-based workflows, create a **.env** file in the example directory and load it in the notebook. Do not hardcode credentials directly in notebook cells.

```
$ cat .env
OPENAI_API_KEY=<your_api_key>
TEACHER_MODEL=openai/gpt-5.2
JUDGE_MODEL=openai/gpt-4o
```

Replace **<your_api_key>** with your actual API key. Add **.env** to your **.gitignore** file to prevent accidental commits.

Kubernetes Secrets

For production deployments on Red Hat OpenShift AI, store credentials as Kubernetes Secrets and mount them as environment variables in your workbench or pipeline pods.

```
apiVersion: v1
kind: Secret
metadata:
  name: mcp-eval-credentials
  namespace: <your_namespace>
type: Opaque
stringData:
  OPENAI_API_KEY: "<your_api_key>"
  LANGGRAPH_API_KEY: "<your_langgraph_key>"
```

where:

<your_namespace>

Specifies the namespace where your workbench runs.

<your_api_key>

Specifies your frontier LLM API key.

<your_langgraph_key>

Specifies your LangGraph agent API key.

Connection API

For platform-integrated credential management, use the OpenShift AI Connection API to store and retrieve credentials.

MCP server authentication

The MCP specification standardizes on OAuth 2.1 with PKCE for remote server authentication. The evaluation pipeline currently supports API key-based authentication through environment variables.



NOTE

The scope of supported MCP server authentication methods include OAuth, API keys, and mTLS

Credential handling practices to avoid

Avoid the following practices when handling credentials for the evaluation pipeline:

- Do not hard code API keys directly in notebook cells. Use environment variables loaded from **.env** files.
- Do not commit **.env** files or notebooks that contain credentials to version control. Add **.env** to your **.gitignore** file.
- Do not log credentials in pipeline output. Verify that your MCP server logging configuration does not include authentication headers in log entries.
- Do not expose secrets in the **MODEL_CONFIGS** dictionary when sharing notebooks. Replace credential values with placeholder references before sharing.

5.4. MCP EVALUATION PIPELINE CONFIGURATION OPTIONS

You can configure the MCP evaluation benchmark pipeline by using environment variables, notebook dictionaries, and startup script flags. The following tables describe all available configuration options.

Environment variables

Table 5.2. Environment variables for the evaluation pipeline

Variable	Required	Default	Description
OPENAI_API_KEY	Yes	None	Specifies the API key for the frontier LLM endpoint used by the teacher and judge models.
TEACHER_MODEL	No	openai/gpt-5.2	Specifies the frontier model used for question generation and ground truth creation.
JUDGE_MODEL	No	openai/gpt-4o	Specifies the model used for LLM-as-judge quality scoring during evaluation. Compatible alternatives include openai/gpt-5.2 and openai/gpt-5-mini . The openai/ prefix is required.
LANGGRAPH_API_KEY	No	None	Specifies the API key for authenticating with LangGraph agent endpoints, if your agents require authentication.

Notebook configuration dictionaries

The generation and evaluation notebooks contain configuration dictionaries that you must update to match your MCP server deployment.

Table 5.3. Configuration dictionaries in the generation notebook

Dictionary	Notebook	Description
------------	----------	-------------

Dictionary	Notebook	Description
MCP_SERVERS	generate.ipynb	Maps server names to MCP server streamable HTTP URLs. For example, {"my_server": "http://localhost:8001"}.
AGENT_URLS	generate.ipynb	Maps server names to LangGraph agent URLs. For example, {"my_server": "http://localhost:2024"}.
SERVER_DESCRIPTIONS	generate.ipynb	Provides a natural language description for each MCP server. The pipeline uses these descriptions to generate contextually relevant evaluation questions.
NUM_SAMPLES_LEVELS	generate.ipynb	Defines the complexity levels for question generation as a list of integers. The default is [2, 4, 8].
MODEL_CONFIGS	evaluate.ipynb	Maps candidate model names to their endpoint configuration, including api_base and api_key fields. Each entry defines a model to evaluate against the generated benchmarks.

Complexity levels

The **NUM_SAMPLES_LEVELS** configuration controls the complexity of generated evaluation questions. Each value in the list represents the number of tools the pipeline attempts to incorporate into a single evaluation question.

Table 5.4. Complexity level mapping

num_samples value	Complexity	Description
2	Simple	Generates questions that require one or two tool calls to answer.
4	Moderate	Generates questions that require two to three tools in sequence, with output from one tool informing the input to the next.
8	Complex	Generates questions that require multi-step tool chaining across many tools, testing dependency awareness and planning.

If a server has fewer tools than the **num_samples** value, that complexity level is automatically skipped for that server. You can customize these levels by modifying the **NUM_SAMPLES_LEVELS** list.

The **num_samples** value is passed to the **sample_tools** block through the **runtime_params** syntax in the SDG Hub flow.

SDG Hub flow names

Table 5.5. Flows used by the evaluation pipeline

Flow name	Block count	Purpose
MCP Server Distillation	23	Handles MCP exploration, question generation, and ground truth generation.
Agent Tool-Use Evaluation	4	Scores candidate models against the generated benchmarks by using LLM-as-judge and programmatic metrics.

Default port assignments

Table 5.6. Default port assignments for MCP servers and LangGraph agents

Component	Port range	Description
MCP servers	8001-8006	Default ports for the example MCP servers started by start_servers.sh .
LangGraph agents	2024-2029	Default ports for the LangGraph agents started by start_agents.sh . Each agent connects to the MCP server with the corresponding index.

5.4.1. Startup script options

The **start_servers.sh** and **start_agents.sh** scripts manage MCP server and LangGraph agent lifecycle.

Table 5.7. Startup script flags

Flag	Description
No flags	Starts all MCP servers or LangGraph agents in the background.
--check	Checks the status of running servers or agents and reports which are healthy.
--stop	Stops all running servers or agents.

Caching behavior

The pipeline caches results per server during generation. If you run the generation notebook again, servers that already have cached results are automatically skipped.

To force regeneration for a specific server, delete the cached output files for that server from the output directory before re-running the notebook.

Runtime parameters

Table 5.8. Additional runtime parameters

Parameter	Default	Description
Timeout	300 seconds	Specifies the maximum time to wait for a single tool invocation or LLM call before timing out. Increase this value if your MCP servers have high-latency tools.

5.5. GENERATE EVALUATION DATA FROM CUSTOM MCP SERVERS

You can generate tool-calling evaluation benchmark data from your custom MCP servers and then evaluate candidate models against those benchmarks. The pipeline produces verified question-answer-tool-call triplets that you use to rank models by their tool-calling accuracy.

Prerequisites

- The SDG Hub library is installed from the Red Hat AI Python index, as described in [Set up your working environment](#).
- One or more custom MCP servers are available and accessible.
- You have access to a frontier LLM endpoint that is compatible with the OpenAI API for teacher and judge models.
- The **OPENAI_API_KEY** environment variable is set for your frontier LLM endpoint.
- The required Python packages are installed. Install them by running **pip install -r requirements.txt** from the **sdg_hub/examples/agentic/mcp_distillation_evaluation** directory, or see the example directory's README for version-specific instructions.
- You have reviewed the [safety considerations for MCP tool exploration](#) and verified that your MCP servers are safe for automated tool invocation.

Procedure

1. Clone the SDG Hub repository and navigate to the MCP evaluation example directory:

```
$ git clone https://github.com/Red-Hat-AI-Innovation-Team/sdg_hub.git
$ cd sdg_hub/examples/agentic/mcp_distillation_evaluation
```

2. Configure the environment variables by creating a **.env** file or exporting values in your terminal:

```
$ export OPENAI_API_KEY="<your_api_key>"
$ export TEACHER_MODEL="<teacher_model>"
$ export JUDGE_MODEL="<judge_model>"
```

where:

<your_api_key>

Specifies your API key for the frontier LLM endpoint.

TEACHER_MODEL

Specifies the frontier model used for question generation and ground truth creation. The default is **openai/gpt-5.2**.

JUDGE_MODEL

Specifies the model used for LLM-as-judge quality scoring. The default is **openai/gpt-4o**.

3. Start your MCP servers using the startup script:

```
$ bash start_servers.sh
```

By default, MCP servers start on ports 8001 through 8006. If you are using your own custom MCP servers, update the **MCP_SERVERS** value in the generation notebook to point to your server URLs.

4. Verify that all MCP servers are running:

```
$ bash start_servers.sh --check
```

The output displays the status of each server. All servers must be running before proceeding. If a server fails to start, check for port conflicts or missing dependencies in the server logs.

5. Deploy LangGraph agents connected to each MCP server:

```
$ bash start_agents.sh
```

One LangGraph agent is deployed per MCP server on ports 2024 through 2029 by default. Each agent connects to a single MCP server and provides the tool-calling interface that the pipeline uses for exploration and evaluation.

6. Verify that all agents are running:

```
$ bash start_agents.sh --check
```

The output shows the status of each agent. All agents must be in a running state before proceeding.

7. Open the **generate.ipynb** notebook and configure the server and agent dictionaries to match your MCP server deployment.

The notebook contains the following configuration dictionaries:

- **MCP_SERVERS**: Maps server names to MCP server URLs.
- **AGENT_URLS**: Maps server names to LangGraph agent URLs.
- **SERVER_DESCRIPTIONS**: Provides context-aware descriptions for each server that the pipeline uses during question generation.

8. Run all cells in the **generate.ipynb** notebook to start the four-stage pipeline as described in the "MCP evaluation benchmark pipeline" documentation.

- To evaluate candidate models against the generated benchmarks, open the **evaluate.ipynb** notebook and configure the **MODEL_CONFIGS** dictionary:

```
MODEL_CONFIGS = {
    "<model_name_1>": {
        "api_base": "<model_endpoint_url_1>",
        "api_key": "<model_api_key_1>"
    },
    "<model_name_2>": {
        "api_base": "<model_endpoint_url_2>",
        "api_key": "<model_api_key_2>"
    }
}
```

where:

<model_name_1>, <model_name_2>

Specifies the name of each candidate model.

<model_endpoint_url_1>, <model_endpoint_url_2>

Specifies the OpenAI API-compatible endpoint URL for each model.

<model_api_key_1>, <model_api_key_2>

Specifies the API key for each model endpoint.

Each candidate model is scored through the same LangGraph agent harness that was used for data generation. The underlying LLM is swapped by using the LangGraph configurable parameter, keeping the full agent stack identical.

- Run the evaluation notebook cells to score each candidate model against the benchmarks. The evaluation produces the following results:

- Per-server scores that show how each model performs on each MCP server's tools.
- LLM-as-judge scoring dimensions: task fulfillment, grounding, tool appropriateness, parameter accuracy, dependency awareness, parallelism and efficiency.
- Trace metrics: tool recall, tool precision, order match, and parameter match.
- Overall model rankings based on composite scores across all servers.

Verification

- Verify that the **benchmark_tasks.jsonl** file exists in the **sdg_hub/examples/agentic/mcp_distillation_evaluation** directory and contains records for each server:

```
$ wc -l benchmark_tasks.jsonl
$ head -1 benchmark_tasks.jsonl | python3 -m json.tool
```

Each record must contain the **server**, **question**, **expert_answer**, **expert_tools**, **expert_tool_trace**, **question_quality_rating**, and **completeness_rating** fields.

- Verify that the **evaluation_results.jsonl** file contains scores for all configured models:

```
$ grep -c "<model_name>" evaluation_results.jsonl
```

■

Replace **<model_name>** with the name of each candidate model to confirm that scores exist for every model.

5.6. CONFIGURE QUALITY VALIDATION FOR EVALUATION DATA

After you complete the initial evaluation, you can tune the quality validation settings to refine how the LLM-as-judge scores candidate models and how results are cached and aggregated. You can change the judge model, adjust scoring parameters, and re-run evaluation with updated settings.

Prerequisites

- You have completed the evaluation data generation procedure, as described in [Generate evaluation data from custom MCP servers and evaluate models](#).
- The **benchmark_tasks.jsonl** file exists in the **sdg_hub/examples/agent/mcp_distillation_evaluation** directory.
- The **JUDGE_MODEL** environment variable is set. The default is **openai/gpt-4o**.
- You have access to the **evaluate.ipynb** notebook in the MCP evaluation example directory.

Procedure

1. Set the judge model by exporting the **JUDGE_MODEL** environment variable:

```
$ export JUDGE_MODEL="openai/gpt-4o"
```

Alternative judge models include: **openai/gpt-5.2** and **openai/gpt-5-mini**. The **openai/** prefix is required for all model names.

The judge model evaluates candidate model responses across six quality dimensions.

2. Verify that the judge temperature is set to **0.0** for deterministic scoring. The default temperature is **0.0**, which produces consistent scores across repeated evaluation runs. If you need to explore score variance, you can increase the temperature, but this reduces reproducibility.
3. Run the Agent Tool-Use Evaluation flow in the **evaluate.ipynb** notebook. The evaluation flow scores each candidate model by running it through the same LangGraph agent harness that was used during data generation. The primary difference is that the underlying LLM is swapped to the candidate model by using the LangGraph configurable parameter. This ensures that evaluation scores reflect the model's tool-calling accuracy.
4. Review the evaluation results by examining the per-server score averages. Each server contributes equally to the overall model ranking, regardless of the number of tasks per server. A model that performs well on servers with many tools but poorly on servers with few tools receives balanced scoring. Check the six LLM-as-judge dimension scores for each model to identify specific weaknesses:
 - Low **task_fulfillment** or **grounding** scores indicate that the model's answers are incomplete or not grounded in tool output.

- Low **tool_appropriateness** or **parameter_accuracy** scores indicate that the model selects incorrect tools or passes wrong parameters.
 - Low **dependency_awareness** or **parallelism_and_efficiency** scores indicate that the model does not plan tool-call sequences effectively.
5. Check the four programmatic trace metrics to identify objective tool-calling failures:
 - Low **tool_recall** indicates that the model missed required tools.
 - Low **tool_precision** indicates that the model called unnecessary tools.
 - Low **order_match** indicates that the model called tools in the wrong sequence.
 - Low **param_match** indicates that the model passed incorrect parameters.
 6. Optional: To force re-evaluation of all models, delete the **evaluation_results.jsonl** file before re-running the notebook.



NOTE

Evaluation results are cached in the **evaluation_results.jsonl** file. If you run the evaluation again with the same models and benchmark tasks, the pipeline uses cached results. The cache is automatically invalidated when the **benchmark_tasks.jsonl** file changes.

Verification

- Confirm that all candidate models have scores across all servers by checking the evaluation results summary in the notebook output. Each model must have scores for every server in the **benchmark_tasks.jsonl** file.
- Check for **ZERO_JUDGE** and **ZERO_METRICS** failure indicators in the evaluation results. These constants indicate tasks where a model failed to produce any valid response. A high proportion of zero-value results for a specific model suggests that it cannot handle the tool-calling tasks.

5.7. MCP EVALUATION OUTPUT FORMAT AND SCORING DIMENSIONS

The evaluation benchmark pipeline produces two JSONL output files: **benchmark_tasks.jsonl** with generated evaluation tasks and **evaluation_results.jsonl** with model scoring results. You can use these files to interpret model performance, compare candidates, and integrate results with downstream evaluation workflows.

5.7.1. Benchmark tasks output schema

The **benchmark_tasks.jsonl** file contains one JSON record per line. Each record represents a single evaluation task.

Table 5.9. Fields in **benchmark_tasks.jsonl**

Field	Type	Description
server	String	The name of the MCP server that this task targets.
question	String	The natural language question that a model must answer by using the MCP server's tools.
expert_answer	String	The verified ground truth answer produced by the frontier model during data generation.
expert_tools	List of strings	The list of tool names that the frontier model invoked to answer the question.
expert_tool_trace	List of objects	The canonical tool trace that records each tool call with name , input , and output keys. This trace serves as the gold standard for evaluation.
question_quality_rating	Float	The LLM-as-judge quality score for the generated question, indicating how well the question tests tool-calling ability.
completeness_rating	Float	The LLM-as-judge completeness score for the ground truth answer, indicating how thoroughly the answer addresses the question.

5.7.2. Evaluation results output schema

The **evaluation_results.jsonl** file contains one JSON record per line. Each record represents the evaluation of a single candidate model on a single benchmark task.

Table 5.10. Identifier fields in **evaluation_results.jsonl**

Field	Type	Description
server	String	The MCP server that the task targets.
model	String	The candidate model being evaluated.
task_idx	Integer	The index of the benchmark task within the server's task set.

5.7.3. Programmatic trace metrics

Four metrics are computed by comparing the candidate model's tool trace against the expert gold-standard trace. All programmatic metrics are on a 0.0 to 1.0 scale.

Table 5.11. Programmatic trace metrics

Metric	Description	Computation
tool_recall	Measures whether the model called all the tools that the expert called.	Set intersection of model tools and expert tools, divided by the expert tool set size.
tool_precision	Measures whether the model avoided calling unnecessary tools.	Set intersection of model tools and expert tools, divided by the model tool set size.
order_match	Measures whether the model called tools in the correct sequence.	Longest common subsequence of model and expert tool lists, divided by the expert list length.
param_match	Measures whether the model passed correct parameters to each tool.	Key overlap multiplied by value similarity across matched tool calls.

5.7.4. LLM-as-judge scoring dimensions

Dimensions are scored by the LLM judge model. All LLM-as-judge scores are on a 1 to 10 integer scale.

Table 5.12. LLM-as-judge scoring dimensions grouped by category

Category	Dimension	Description
Task Completion	task_fulfillment	Measures how completely the model answered the question.
Task Completion	grounding	Measures whether the answer is grounded in actual tool output rather than fabricated.
Tool Selection	tool_appropriateness	Measures whether the model selected the correct tools for the task.
Tool Selection	parameter_accuracy	Measures whether the model passed correct parameters to each tool.
Planning	dependency_awareness	Measures whether the model respected tool dependencies, for example, calling a lookup tool before a detail tool.
Planning	parallelism_and_efficiency	Measures whether the model used efficient tool-calling patterns, such as parallel calls when dependencies allow.

5.7.5. Defect-rate calibration rubric

The LLM-as-judge uses a defect-rate calibration rubric to ensure consistent scoring. The rubric maps score ranges to expected defect rates in the model's response.

Table 5.13. Defect-rate calibration rubric for LLM-as-judge scores

Score range	Defect rate	Description
9-10	0-10%	Near-perfect performance with minimal or no defects. Reserved for exceptional cases.
7-8	10-30%	Minor defects that do not materially affect the answer quality. Requires evidence of strong performance.
5-6	30-50%	Moderate defects. Some tool calls or parameters are incorrect but the overall approach is reasonable.
3-4	50-70%	Significant defects. Multiple incorrect tool selections or parameters.
1-2	70-100%	Major defects. The model fails to use tools correctly or produces an ungrounded answer.

The default baseline score is 4 to 5. Scores of 8 or higher are reserved for exceptional performance and require strong evidence from the tool trace.

5.7.6. Failure case constants

When a model fails to produce a valid response for a task, the pipeline assigns predefined zero-value constants.

ZERO_JUDGE

All six LLM-as-judge dimensions are set to 1, which is the minimum score.

ZERO_METRICS

All four programmatic metrics are set to 0.0.

Check for these constants in your evaluation results to identify tasks where models failed completely.

5.8. MCP EVALUATION FRAMEWORK COMPATIBILITY

You can integrate the evaluation benchmark output with external evaluation frameworks to extend your model validation workflow. Each framework has specific requirements for consuming the pipeline's JSONL output.

lm-evaluation-harness integration

The **lm-evaluation-harness** framework does not natively support tool-calling evaluation. To evaluate models with the generated benchmark data set, you must create a custom task YAML definition that uses the **local-completions** backend.

The custom task YAML defines how lm-eval reads the **benchmark_tasks.jsonl** file, extracts the question and ground truth fields, and scores model responses. Because tool-calling requires structured function-call output rather than simple text completion, the task definition must include custom processing logic to parse and compare tool traces.

For guidance on creating custom task YAML definitions, see the [lm-evaluation-harness custom task documentation](#).

EvalHub integration

EvalHub collections are structured benchmark configurations loaded through YAML or ConfigMap. The evaluation pipeline output is not directly importable into EvalHub without a custom adapter.

To use the generated benchmarks with EvalHub, you must create an adapter that transforms the **benchmark_tasks.jsonl** format into a benchmark configuration that an EvalHub provider can process. The adapter must:

- Define a custom EvalHub provider that understands tool-calling evaluation tasks
- Map the benchmark task fields such as **question**, **expert_tools**, and **expert_tool_trace** to the provider's expected input format
- Register the provider and benchmark configuration with the EvalHub server

For more information about EvalHub providers and collections, see [EvalHub evaluation orchestration service](#).

mcp-bench alignment

The six LLM-as-judge scoring dimensions used by the SDG Hub evaluation pipeline align with the two-tier evaluation framework used by mcp-bench. Both frameworks combine programmatic tool-trace comparison with LLM-as-judge scoring across similar quality dimensions.

In initial validation testing, the SDG Hub pipeline achieved a rank correlation of 1.000 against mcp-bench across 7 models on 111 tasks. This correlation means that the pipeline produces model rankings that are consistent with the mcp-bench reference implementation for the tested configuration. Correlation might vary with different model sets, task compositions, or MCP server configurations.

The key difference between the SDG Hub pipeline and mcp-bench is that the SDG Hub pipeline generates evaluation data from your custom MCP servers, while mcp-bench provides a fixed benchmark suite. You can use the SDG Hub pipeline to produce domain-specific benchmarks for your tools, and compare the resulting model rankings against mcp-bench rankings as a validation baseline.

CHAPTER 6. TRAIN THE MODEL BY USING YOUR PREPARED DATA

To train the model, you can use the Red Hat Training Hub and the Kubeflow Training Operator (KFTO).

You can simplify and accelerate the process of fine-tuning and customizing a foundation model by using your own data. The Red Hat Training Hub is an algorithm-focused interface for common LLM training, continual learning, and reinforcement learning techniques.

6.1. EXPLORE TRAINING HUB EXAMPLES

The Training Hub repository hosts multiple cookbooks for using different LLM algorithms such as Supervised Fine-tuning (SFT), Orthogonal Subspace Fine-tuning (OSFT)/Continual Learning, and Low-Rank Adaptation (LoRA)/Quantized Low-Rank Adaptation (QLoRA). OSFT is a training algorithm built by the Red Hat AI Innovation team. With OSFT, you can continually post-train a fine-tuned model to expand its knowledge on new data. You can tinker with the Training Hub cookbooks from a workbench within your OpenShift AI project.

To get started with Training Hub, explore the provided examples.

Prerequisites

- Install the Training Hub library as described in [Set up your working environment](#).

Procedure

1. To access Training Hub examples, clone the [Training Hub Git repository](#):
 - To clone the https://github.com/Red-Hat-AI-Innovation-Team/training_hub.git repository from JupyterLab, follow the steps in [Clone an example Git repository](#).
 - To create a local clone of the repository, run the following command:


```
git clone https://github.com/Red-Hat-AI-Innovation-Team/training_hub
```
2. Go to the [examples](#) directory to view Training Hub notebooks, Python scripts, and documentation.
 - For a quick overview and descriptions of the supported algorithms and features, with links to examples and getting started code, see the top-level **README** file.
 - For detailed parameter documentation, see the **docs** directory.
 - For hands-on learning with the interactive notebooks, see the **notebooks** directory.
 - For pre-written, configurable python scripts to run training algorithms with various language models, see the **scripts** directory.

6.2. TRAINING HUB ALGORITHM AND MODEL SUPPORT MATRIX

To simplify tuning for enterprise customers, Training Hub supports multiple backends and exposes a unified API surface to access the latest training algorithms from different backends.

The following table lists Training Hub algorithm and model support matrix.

Table 6.1. Training Hub algorithm and model support matrix

Algorithm	Backend	Supported Model Architectures
Supervised Fine-tuning (SFT)	instructlab.training	GPTossForCausalLM (GPT OSS 20B/120B) LlamaForCausalLM (Llama 3 Models) Qwen2ForCausalLM (Qwen 2.5 models) Qwen3ForCausalLM (Qwen 3 models) GraniteForCausalLM (Granite 3 models) GraniteMoeHybridForCausalLM (Granite 4 models) Phi3ForCausalLM (Phi 3 and 4 models) MistralForCausalLM (Mistral models)
Orthogonal Subspace Fine-tuning (OSFT)	mini-trainer	Same as SFT
Low-Rank Adaptation (LoRA) /Quantized Low-Rank Adaptation (QLoRA)	Unsloth	GPTossForCausalLM (GPT OSS 20B/120B) (QLoRA ONLY) LlamaForCausalLM (Llama 3 Models) Qwen2ForCausalLM (Qwen 2.5 models) Qwen3ForCausalLM (Qwen 3 models) GraniteForCausalLM (Granite 3 models) GraniteMoeHybridForCausalLM (Granite 4 models) MistralForCausalLM (Mistral models)

NOTE: If you experience an issue with the model classes listed for OSFT with **use_liger=True**, try setting **use_liger=False**. Liger kernels are supported for most model architectures, but some newer architectures might experience errors or instability if not fully supported. For up-to-date support information, see the [Liger-Kernel GitHub repository](#).

6.3. ESTIMATE MEMORY USAGE

To learn how to estimate the amount of memory you need for running and training a specific model, as well as whether your configured GPUs can support the model, use the memory estimator. The memory

estimator currently supports only Supervised Fine-tuning (SFT) and Orthogonal Subspace Fine-tuning (OSFT) algorithms. See the following example files in the [Training Hub Git repository](#):

- For the Memory Estimator API, see the **src/training_hub/profiling/memory_estimator.py** file.
- For an example notebook that uses the API, see **notebooks/memory_estimator_example.ipynb** file.

6.4. COMPARE THE PERFORMANCE OF OSFT, SFT, AND LORA TRAINING ALGORITHMS

You can use the Orthogonal Subspace Fine-Tuning (OSFT), Supervised Fine-Tuning (SFT), and Low-Rank Adaptation (LoRA) algorithms in Training Hub.

Use SFT to fine-tune a model on supervised data sets with support for:

- Single-node and multi-node distributed training
- Configurable training parameters, for example, epochs, batch size, and learning rate.
- InstructLab-Training backend integration

Use OSFT to fine-tune a model while controlling how much of its existing behavior to preserve, with support for:

- Single-node and multi-node distributed training
- Configurable training parameters (for example, epochs, batch size, learning rate)
- RHA Innovation Mini-Trainer backend integration

Use LoRA for parameter-efficient fine-tuning with significantly reduced memory requirements, with support for:

- Training low-rank adaptation matrices instead of full model weights
- Unsloth backend integration
- QLoRA variant for further memory reduction (Float4)

The [examples/docs](#) directory contains information and examples for how to use each algorithm.

Here is a performance comparison of using OSFT, SFT, and LoRA in Training Hub.

NOTE: When scaling the usage of Liger Kernels for all methods, some amount of fixed overhead memory is added to all methods that do not use Liger Kernels.

- **Memory scaling:** OSFT adds additional memory overhead to the model storage due to its unique matrices, roughly about 1.25-1.5x that of the normal model storage in SFT. However, the rest of OSFT memory scales linearly with the unfreeze rank ratio (URR). The URR is a hyperparameter for OSFT that is a value between 0 and 1. It represents the fraction of the matrix rank that is unfrozen and updated during fine-tuning.
A rough comparison is: OSFT Memory $\sim 3 \times r \times$ SFT Memory, where r is the URR unfreeze rank ratio, the fraction of the matrix being fine-tuned. At URR = 1/3, OSFT and SFT have similar memory usage.

In most post-training setups, URR values below 1/3 are sufficient for learning new tasks, making OSFT notably lighter in memory.

Like SFT, LoRA requires a fixed amount of overhead memory to store the base model, intermediate activations, and outputs. The rest of the memory needed for LoRA scales linearly based on the LoRA rank (**lora_r**) parameter. The **lora_r** value is an integer, ideally no more than the size of any of the model's weight dimensions, that determines how many rows should be used in each of LoRA's approximated matrices.

You should keep the **lora_r** value as low as possible. As **lora_r** approaches 0, the memory that LoRA uses should approach $1/4 * \text{SFT}$. While it is difficult to precisely compare SFT and LoRA, LoRA's memory usage should begin to reach or exceed that of SFT's if the value of **lora_r** is more than $3/8$ of the size of the hidden dimensionality. Note that the memory used by LoRA in Training Hub is further reduced by the fact that LoRA uses Float16 as its main datatype. QLoRA uses Float4 instead. Note that when using QLoRA, you must briefly place the Float16 model onto the GPU, which can bottleneck memory usage.

- **Training time:** On data sets of equal size, OSFT typically takes about twice as long per phase. However, because OSFT does not require replay buffers from past tasks, unlike SFT, the total training time across multiple phases or tasks is lower with clear benefits as the number of tasks grows. Because OSFT supports continual learning without maintaining or reusing old data, it enables lighter, single-pass end-to-end runs.

6.5. TRAINING HUB IN OPENSIFT AI

On OpenShift AI, you can run Training Hub training in an interactive mode where training runs directly in your notebook environment. This single in-workbench pod approach gives you fast iteration for small experiments, immediate feedback during development, and easy debugging. You can inspect variables, logs, and intermediate artifacts in real time.

To run in-workbench training, use the **Training | Jupyter | PyTorch | CUDA | Python** notebook image. This image is pre-packaged with all the libraries and dependencies required by Training Hub. All included packages are curated and distributed through the Red Hat package index, ensuring they are tested, supported, and kept up to date. You can start training without installing additional packages.

For comprehensive tutorials on Fine Tuning with Training Hub, follow these guided examples:

- [SFT Fine-Tuning with Kubeflow Training on OpenShift AI](#)
- [OSFT Continual Learning on Red Hat OpenShift AI](#)
- [LoRA/QLoRA Fine-Tuning with Training Hub](#)

6.6. TRACK EXPERIMENTS WITH MLFLOW AND TRAINING HUB

Training Hub provides automatic experiment tracking integration with MLflow. You can track, compare, and manage machine learning experiments across different training runs and algorithms. MLflow is an open-source platform for managing the ML lifecycle, including experimentation, reproducibility, and deployment.

Training Hub automatically logs the following information to MLflow:

- All training hyperparameters, such as learning rate, batch size, and number of epochs.
- Training metrics, such as loss, learning rate schedule, and gradient norms.

- Model configuration and architecture details.
- Run metadata, such as timestamps, duration, and hardware configuration.



NOTE

In distributed training for multi-GPU or multi-node, to avoid duplicate entries, only rank 0 performs logging.

The integration works alongside other logging systems, such as Weights & Biases, TensorBoard, and JSONL logging.

6.6.1. Benefits of experiment tracking

Experiment tracking eliminates the need to manually track hyperparameters, metrics, and results in spreadsheets or notebooks. With MLflow integration, Training Hub automatically logs all training parameters and metrics. You can use the logs to help you with the following tasks:

- * Compare performance across different model configurations with standardized logging
- * Maintain a complete record of all training runs with full reproducibility context
- * Share experimental results with team members through live dashboards instead of files or screenshots
- * Make data-driven decisions for hyperparameter selection based on easy comparison across runs
- * Accelerate experimentation velocity by eliminating manual bookkeeping overhead.

6.6.2. Enable MLflow tracking

Training Hub supports MLflow tracking for all training algorithms: Supervised Fine-tuning (SFT), Orthogonal Subspace Fine-tuning (OSFT), and Low-Rank Adaptation (LoRA).

Prerequisites

- You have access to an MLflow tracking server.
- You can run MLflow locally or use an enterprise MLflow deployment.

Procedure

1. To enable MLflow tracking, provide the MLflow tracking URI when you call a training function as shown in the following example:

```
from training_hub import sft

sft(
    model_path="Qwen/Qwen2.5-7B-Instruct",
    data_path="./data.jsonl",
    ckpt_output_dir="./checkpoints",
    num_epochs=3,
    learning_rate=2e-5,
    mlflow_tracking_uri="http://localhost:5000",
    mlflow_experiment_name="qwen-sft-experiments",
    mlflow_run_name="baseline-run"
)
```

2. Optional. Configure MLflow tracking by using API parameters or environment variables as listed in the following table:

Configuration description	API Parameter	Environment Variable	Required?
The URI of the MLflow tracking server. Training Hub automatically enables MLflow logging without requiring additional configuration.	mlflow_tracking_uri	MLFLOW_TRACKING_URI	Required. The workbench exposes the MLFLOW_TRACKING_URI environment variable.
The name of the MLflow experiment to log runs under	mlflow_experiment_name	MLFLOW_EXPERIMENT_NAME	Optional
The name for this specific training run	mlflow_run_name	MLFLOW_RUN_NAME	Optional

Additional resources

For detailed parameter documentation and advanced configuration options, see the [Training Hub documentation](#) in the Training Hub repository.

6.7. DISTRIBUTE TRAINING JOBS BY USING THE KUBEFLOW TRAINER

If you want to implement distributed training across multiple nodes to meet the needs of your training workloads, you can use the Kubeflow Trainer. The Kubeflow Trainer abstracts the underlying infrastructure complexity of distributed training and fine-tuning of models. The iterative process of fine-tuning significantly reduces the time and resources required compared to training models from scratch.

Learn more about the Kubeflow Trainer in the following OpenShift AI documentation:

- [Running Training Operator-based distributed training workloads](#) in the *Working with distributed workloads* guide.

6.7.1. Distributed fine-tuning with Training Hub and Kubeflow Trainer

The Kubeflow Trainer supports distributed fine-tuning by using Training Hub, abstracting the complexity of distributed training. It seamlessly manages scaling and orchestration for you, allowing you to focus on your domain-specific fine-tuning logic by using the simplified Training Hub APIs.

For a comprehensive tutorial on Fine Tuning with Training Hub leveraging distributed nodes with the Kubeflow Trainer, follow these guided examples:

- [SFT Fine-Tuning with Kubeflow Training on OpenShift AI](#)
- [OSFT Continual Learning on Red Hat OpenShift AI](#)
- [LoRA/QLoRA Fine-Tuning with Training Hub](#)

CHAPTER 7. IMPROVE MODEL ACCURACY WITH INFERENCE-TIME SCALING

Inference-time scaling (ITS) improves the quality of model responses by allocating additional compute at inference time. Instead of generating a single response to a prompt, the model generates multiple candidate outputs and uses a selection strategy to return the best one. This approach produces better results without modifying model weights, at the cost of increased compute per query.

The ITS Hub library provides scaling algorithms that you can use with any model accessible through an OpenAI-compatible API, including models served with vLLM on Red Hat OpenShift AI.

7.1. GENERATE COMPLETE RESPONSES

The following procedure walks you through using ITS Hub to get better results from a language model without modifying the model itself. You run the following inference-time scaling algorithm against a model served through an OpenAI-compatible API:

- Self-Consistency: Generates multiple responses and picks the most common answer.
- Best-of-N: Generates multiple responses, scores each one, and picks the highest-scoring response.

These two algorithms work with any OpenAI-compatible endpoints and require no additional infrastructure.

Prerequisites

- Python 3.11 or later.
- Access to one of the following OpenAI-compatible model endpoints:
 - A model served with vLLM on Red Hat OpenShift AI.
 - Any other OpenAI-compatible API (for example, OpenAI, Azure OpenAI, or a local vLLM instance).
- The model endpoint URL, API key, and model name.

Procedure

1. Install ITS Hub

```
pip install its-hub[lm]
```

2. Verify the installation:

```
python -c "from its_hub import OpenAICompatibleLanguageModel, SelfConsistency, BestOfN, LLMJudge; print('OK')"
```

3. Connect to your model by creating a language model instance that points to your endpoint. In the following example, replace the endpoint, **api_key**, and **model_name placeholder** values with your actual endpoint details. If your endpoint does not require authentication, as is common with a local vLLM, set **api_key="NO_API_KEY"**.

■

```
from its_hub import OpenAICompatibleLanguageModel

lm = OpenAICompatibleLanguageModel(
    endpoint="https://<your-endpoint>/v1",
    api_key="<your-api-key>",
    model_name="<your-model-name>",
)
```

4. Use the Self-Consistency algorithm to generate multiple responses to the same prompt and select the most common answer. It is similar to asking the same question several times and going with the majority answer.

- **Basic usage**

In the following example, the budget parameter controls how many responses are generated. A budget of 5 means the model generates 5 responses and votes on the answer.

```
from its_hub import SelfConsistency

sc = SelfConsistency()
result = sc.infer(lm, "What is the capital of France?", budget=5)
print(result)
```

- Extract a specific part of the response for voting

By default, Self-Consistency compares the entire response text. Optionally, you can use a projection function to extract a specific part of the response, such as the final answer, as shown in the following example:

```
def extract_last_line(response):
    return response.strip().split("\n")[-1]

sc = SelfConsistency(consistency_space_projection_func=extract_last_line)
result = sc.infer(lm, "What is 2 + 2?", budget=5)
print(result)
```

- Get the full result

By default, **infer()** returns only the winning response as a dictionary. To see all responses and vote counts, set **return_response_only=False**:

```
result = sc.infer(lm, "What is the capital of France?", budget=5,
return_response_only=False)

print(f"Selected response index: {result.selected_index}")
print(f"Vote counts: {result.response_counts}")
print(f"Total responses: {len(result.responses)}")
print(f"Winning response: {result.the_one}")
```

5. Use the Best-of-N algorithm to generate multiple candidate responses, score each one, and return the highest-scoring response. ITS Hub includes an inline implementation of an outcome reward model (LLMJudge).

- Basic usage. In the following example, a budget of 4 means that the model generates 4 candidate responses. The LLM judge scores each one and returns the best.

```
from its_hub import BestOfN, LLMJudge
```

```

judge = LLMJudge(lm=lm)
bon = BestOfN(orm=judge)

result = bon.infer(
    lm,
    "Explain how a CPU works to someone with no technical background",
    budget=4,
)
print(result)

```

- Get the full result:

```

result = bon.infer(
    lm,
    "Explain how a CPU works to someone with no technical background",
    budget=4,
    return_response_only=False,
)

print(f"Scores: {result.scores}")
print(f"Selected index: {result.selected_index}")
print(f"Best response: {result.the_one}")

```

6. Clean up resources:

- To release HTTP connections, run the following command to close the language model instance:

```

import asyncio
asyncio.run(lm.close())

```

- To handle cleanup automatically, use an async context manager:

```

async with OpenAICompatibleLanguageModel(
    endpoint="https://<your-endpoint>/v1",
    api_key="<your-api-key>",
    model_name="<your-model-name>",
) as lm:
    result = await sc.ainfer(lm, "What is the capital of France?", budget=5)

```

7.2. STEP-BY-STEP REASONING

ITS Hub includes algorithms that go beyond generating complete responses. These algorithms build solutions one step at a time, using a process reward model (PRM) to evaluate each reasoning step and steer the generation toward better paths.

- **Beam Search:** Explores multiple reasoning paths in parallel, keeping only the most promising ones at each step.
- **Particle Filtering:** Maintains a diverse set of candidate solutions and uses probabilistic resampling to focus on promising directions.

- Entropic Particle Filtering: An advanced variant that prevents the algorithm from converging too early, which is useful for problems that require many reasoning steps.

These algorithms require a process-reward model with additional infrastructure setup and the experimental installation (`pip install its-hub[experimental]`). They are particularly effective for mathematical reasoning and multi-step problem solving.

For setup instructions and usage examples, see the following resources on ITS Hub docs:

https://github.com/Red-Hat-AI-Innovation-Team/its_hub

- For advanced configuration and all available algorithms, see the algorithm reference.
- To manage concurrency when handling multiple requests, see the orchestration guide.

CHAPTER 8. END-TO-END MODEL CUSTOMIZATION WORKFLOW

You can implement end-to-end workflows by using notebooks and the Kubeflow Trainer for distributed training or by using AI pipelines.

- **Notebook workflow examples**

For a comprehensive notebook tutorial that demonstrates an AI/ML workflow, see the [Knowledge Tuning example](#) on the Red Hat AI examples site.

The Knowledge Tuning tutorial is a curated collection of Jupyter notebooks that includes examples of using Docling to process data, Training Hub to fine-tune a model on that data, and KServe to deploy the final model for a Question and Answer application.

- **Kubeflow Pipeline example**

[This Kubeflow Pipeline example](#) shows how to use Kubeflow pipelines to automate the steps in the Knowledge Training example. By using pipelines, you can run long training jobs or retrain your models on a schedule without having to manually run them in a notebook.

- **AI pipeline example**

You can run an end-to-end model customization workflow by using a fine-tuning AI pipeline, as shown in the [Fine-tuning pipelines on Red Hat OpenShift AI guided example](#). The AI pipelines in this example use [Training Hub algorithms](#) to fine-tune a model, evaluate it, and register it.

CHAPTER 9. SUPPORT PHILOSOPHY: A SECURE PLATFORM

Our primary goal is to provide a secure and reliable platform for serving and customizing models on Red Hat OpenShift AI.

The Python packages for model customization (such as **docling**, **sdg-hub**, and **training-hub**) are key components of this platform.

Our support strategy is focused on the integrity of the platform and the secure delivery of these tools, rather than providing direct, standalone support for the individual Python packages themselves.

What is supported

- **Installation on OpenShift AI:** We fully support the successful installation of these packages from the Red Hat AI Python index onto a supported Red Hat OpenShift AI environment when you use the provided base images.
- **The Platform:** The underlying Red Hat OpenShift AI platform, including its components and infrastructure, is fully supported according to its own lifecycle policy.

What is not supported

- Issues arising from the use of these packages, for example, to build custom flows or applications.
- Mixing packages outside of the packages provided with the Red Hat AI Python Index base images.

The primary benefit of this strategy is a secure software supply chain. By using the Red Hat AI Python Index, you are guaranteed:

- **Red Hat Builds:** You are using Red Hat builds of Python libraries built and delivered by Red Hat and our partners. These builds ensure provenance because Red Hat pulls, scans, and builds all dependencies for the packages.
- **Trusted Source:** The index provides a trusted, secure, and reliable source for your generative AI workflows, especially critical for disconnected (air-gapped) environments.
- **Platform Integrity:** You can be confident that the tools are tested and intended for use on the Red Hat OpenShift AI platform.

For deeper technical questions or contributions related to the packages themselves, we encourage users to engage with the upstream open-source communities.